Reproducibility in Practice

Week 9: CS 203 - Software Tools and Techniques for AI

Prof. Nipun Batra

IIT Gandhinagar

The Friday Night Nightmare

11:30 PM — You (on WhatsApp):

> *"Done! Model hit 94% accuracy. Just pushed* `model_final_v2.py`*. We are going to ace this."*

11:45 PM — Your partner:

> *"I pulled it. I'm only getting 81%."*

11:47 PM — You:

> *"Wait, let me run it again... I'm getting 86% now?! What hyperparameters did I use an hour ago?"*

11:55 PM — Your partner:

> *"I tried to fix it. Now I get* `ModuleNotFoundError: No module named 'xgboost'`*.*

Three Problems, Three Solutions

Every reproducibility failure comes from losing control of one of three things:

Problem	Symptom	Solution
The Machine	"It works on my laptop but not yours"	Docker
The Math	"I get different results every run"	Seeds & determinism
The Memory	"Which of my 50 runs was the good one?"	Experiment tracking (TrackIO)

Today we fix all three — starting with the biggest headache.

The Machine

Docker: Ship the Whole Computer

"Works on My Machine" — The Most Famous Lie in CS

You shared your code AND `requirements.txt`. Your friend installs everything.

```
$ python train.py
OSError: libgomp.so.1: cannot open shared object file
```

What happened? Your code depends on a system library on your Mac that doesn't exist on their Windows laptop.

Virtual environments isolate **Python packages**. They **don't** isolate:

- Operating system (Mac vs Windows vs Linux)
- System libraries (C compilers, CUDA)
- Python version itself

We need to ship the entire computer, not just the package list.

The OYO Room Analogy

Your Hostel Room (Without Docker)

Your ML project lives on your specific laptop — your Python version, your libraries, your OS quirks. If a friend wants to run your code, they have to recreate your entire setup. Usually, it crashes.

The OYO Room (With Docker)

You write a blueprint: *"I need Python 3.10, scikit-learn, my app.py, and my model."*
Your friend runs one command → an identical, fully-furnished room appears on their laptop. App runs perfectly. When they're done, the room vanishes. No mess left behind.

Without Docker	With Docker
45 mins of `pip install` errors	`docker run spam-app` (2 min)
"Which sklearn version?"	Frozen inside the container

Why Docker? Four Scenarios

1. The Assignment Defense

You submit your ML assignment. TA runs it on Windows → crash. With Docker: TA runs `docker run your-project` → guaranteed identical results.

2. The Dependency Minefield

You want to try a cool GitHub repo requiring Python 3.7 + old PyTorch. Installing it will break your other projects. Docker: isolated bubble, no damage.

3. The Group Project Peacemaker

4 teammates: Windows, Mac Intel, Mac Silicon, Linux. Getting a shared pipeline to work across all four? Docker gives everyone the exact same environment.

4. The Portfolio Builder

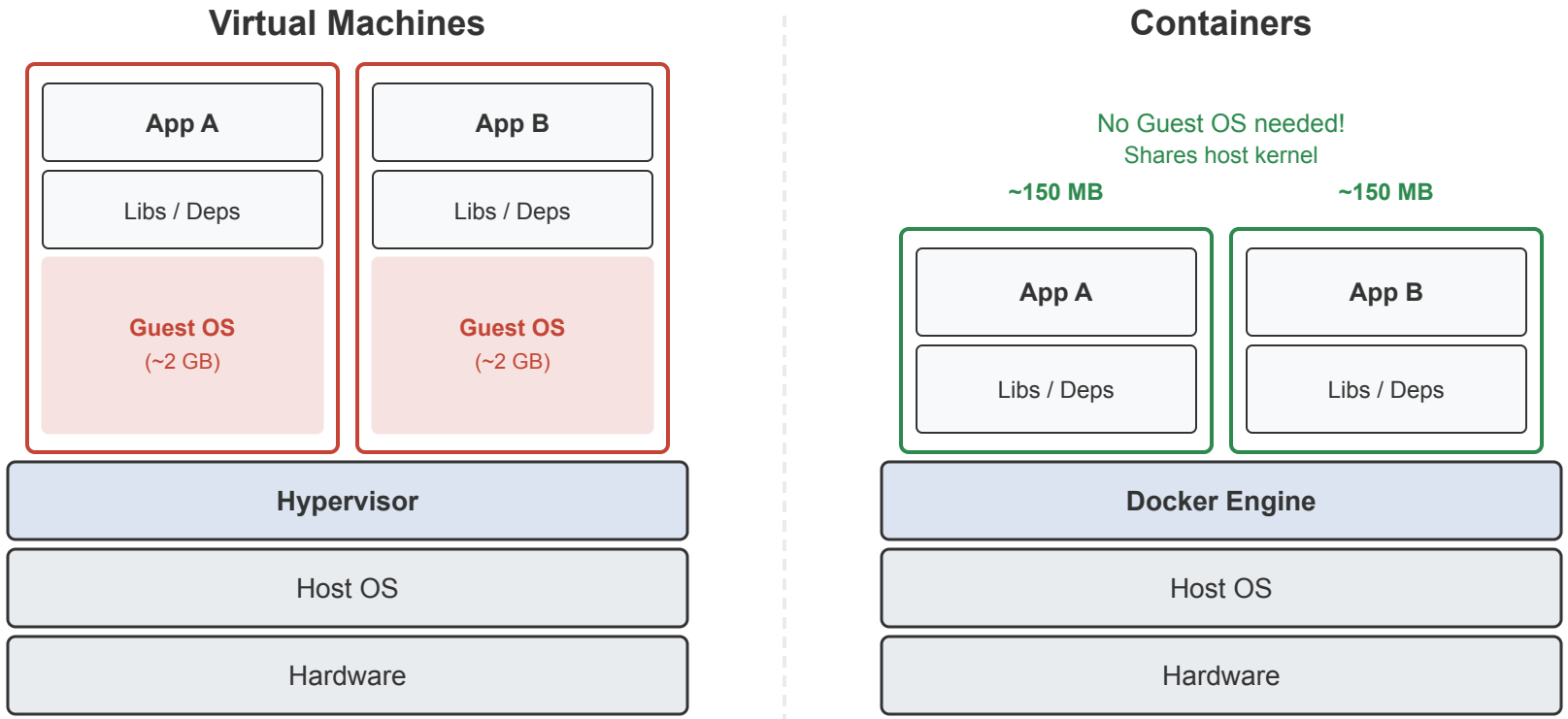
You built a Gradio demo and want to deploy it on HuggingFace Spaces or AWS. Cloud providers want a Docker image, not your laptop.

Six Concepts — That's All

Concept	What It Is	Analogy
Image	Blueprint — OS + libraries + code (read-only)	Recipe card
Container	Running instance of an image	Dish being cooked
Dockerfile	Instructions to build an image	The recipe
Docker Hub	Registry of pre-built images	GitHub for images
Volumes	Shared folder between container and your laptop	USB drive
Ports	How you access the container's web apps	Window into the kitchen

Docker vs Virtual Machines

"Isn't Docker just a VM?"



	VM	Docker Container

Docker Hub: GitHub for Images

`FROM python:3.10-slim` — where does this come from?

Docker Hub (hub.docker.com) — a registry of pre-built images.

```
# Pull an image without building anything
docker pull python:3.10-slim

# See what's available
docker images
```

Image	What's Inside	Size
`python:3.10`	Full Debian + Python + compilers	~1 GB
`python:3.10-slim`	Minimal Debian + Python	~150 MB
`python:3.10-alpine`	Alpine Linux + Python	~50 MB

The Dockerfile: Line by Line

Our Gradio spam classifier needs this `Dockerfile`:

```
FROM python:3.10-slim                # 1. Start with Python + Linux

WORKDIR /app                         # 2. Create a workspace folder

COPY requirements.txt .              # 3. Bring the ingredient list
RUN pip install -r requirements.txt # 4. Install packages

COPY app.py spam_model.pkl ./        # 5. Bring our code + model

ENV GRADIO_SERVER_NAME="0.0.0.0"     # 6. Let the outside world connect

EXPOSE 7860                          # 7. Open the port

CMD ["python", "app.py"]             # 8. What runs when container starts
```

Line 6 is critical: Gradio defaults to `127.0.0.1` (only the container can see itself).

Layer Caching: Why Second Build is Fast

Docker caches each step. If nothing changed, it reuses the cache:

```
$ docker build -t spam-app .
=> CACHED [1/5] FROM python:3.10-slim      # already downloaded
=> CACHED [2/5] WORKDIR /app               # nothing changed
=> CACHED [3/5] COPY requirements.txt .    # file unchanged
=> CACHED [4/5] RUN pip install ...        # packages cached!
=> [5/5] COPY app.py spam_model.pkl ./     # code changed -> rebuilds
```

First build: 2 minutes (downloads everything).

Second build: 5 seconds (only re-copies your code).

Pro tip: That's why we `COPY requirements.txt` before `COPY . .` — if only your code changed, Docker skips the slow `pip install` step!

Walkthrough: Build & Run

> **Demo**: `cd lecture-demos/week09/docker-demo/1-hello`

Build the image:

```
$ docker build -t hello-docker .
```

Run it:

```
$ docker run hello-docker
```

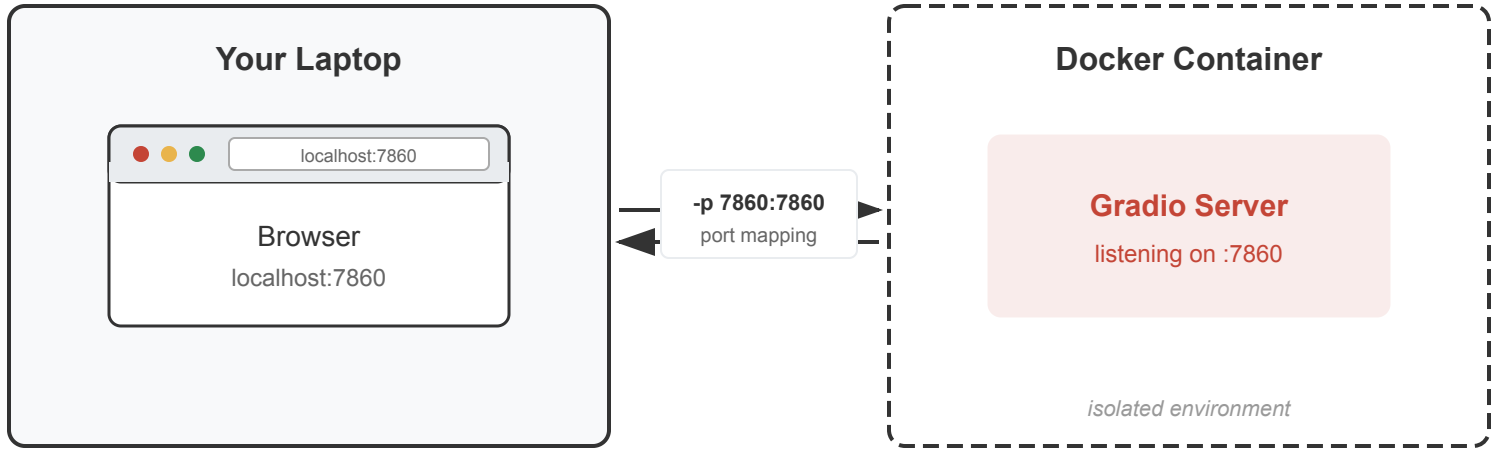
What students see: Python version and OS are different from their laptop. It says "Linux" even on a Mac! This is a completely isolated environment.

> Then: `cd ../2-dependencies` — pinned sklearn, same accuracy every time.
> Then: `cd ../3-web-app` — Gradio app with port mapping.

Port Mapping: Looking Into the Container

```
$ docker run -p 7861:7860 spam-app
```

The `-p 7861:7860` is a **port tunnel** connecting your laptop to the container:



Containers are **headless** — no screen, no browser. You look through the port window.

> **Demo:** `cd 3-web-app` → open `localhost:7861`

Volumes: Containers Have Amnesia

Stop a container → everything inside vanishes. By design!

The fix: Mount a **volume** — like plugging a USB drive into the container.

```
# WITHOUT volume — model disappears when container stops
docker run train-save

# WITH volume — model saved to your laptop
docker run -v $(pwd)/outputs:/app/outputs train-save
ls outputs/   # model.pkl, training_log.txt — survived!
```

`-v $(pwd)/outputs:/app/outputs` = sync this folder between laptop and container.

> **Demo**: `cd 4-volumes` — run with and without `-v`

Environment Variables: Configure Without Rebuilding

Same image, different behavior — using `-e`:

```
# Default: RandomForest with 100 trees
docker run -p 7861:7860 env-demo

# Switch to SVM — no rebuild!
docker run -p 7861:7860 -e MODEL_TYPE=svm env-demo

# Bigger forest — no rebuild!
docker run -p 7861:7860 -e N_ESTIMATORS=500 env-demo
```

In Python: `os.environ.get("MODEL_TYPE", "rf")`

In Dockerfile: `ENV MODEL_TYPE="rf"`

This is how production apps are configured — DB URLs, API keys, feature flags.

> **Demo**: `cd 5-environment`

Common Docker Errors

1. **`docker: command not found`**

→ Docker Desktop isn't running. Open the app first!

2. **`port is already allocated`**

→ Something else is using that port. Use a different one: `-p 7861:7860`

3. **`COPY failed: file not found`**

→ The file isn't in the same folder as your Dockerfile. Check with `ls`.

4. **Image is 3GB!**

→ You used `FROM python:3.10` (full Debian). Use `FROM python:3.10-slim` (~150MB).

```
docker system prune   # clean up old images eating your disk
```

The 6 Docker Commands You Need

```
docker build -t my-app .                # Build image from Dockerfile
docker run my-app                        # Run (foreground, no ports)
docker run -d -p 7861:7860 my-app       # Run (background + port mapping)
docker ps                                # List running containers
docker stop <id>                         # Stop a container
docker exec -it <id> bash               # Shell into running container
```

That's it. Docker solves the machine. But even inside the same Docker container, you can still get different results every run...

The Math

Seeds & Determinism

"Yesterday 92%, Today 85%, Same Code?!"

Run your model training twice — exact same code:

```
X_train, X_test, y_train, y_test = train_test_split(X, y)
model = RandomForestClassifier()
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
```

Run 1: 0.92 **Run 2:** 0.85 **Run 3:** 0.88

Which result do you report? The problem: **uncontrolled randomness.**

Sources of randomness in ML:

- **Train/test split** — which samples go where
- **Model initialization** — starting weights (neural nets)
- **Data shuffling** — order during training

The Minecraft Seed

In Minecraft, the world is randomly generated. But enter the **same seed** → get the **exact same world** every time.

Minecraft	Machine Learning
Seed number	`random_state=42`
Same seed → same world	Same seed → same split, same model
Share seed with friend → same map	Share seed with TA → same results

A random seed locks the universe into one specific timeline.

How to Lock the Universe

In sklearn: set `random_state` everywhere

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier(n_estimators=100, random_state=42)

kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

Global seed (for larger projects):

```
import random, numpy as np

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)

set_seed(42)  # call once at the top of your script
```

Seeds: The Limits

Setting seeds makes YOUR code deterministic. But results can still differ across machines because of:

- **Different OS** → different BLAS/LAPACK math libraries under the hood
- **Different library versions** → sklearn 1.2 vs 1.4 may split data differently
- **GPU non-determinism** → CUDA operations are not always reproducible

That's why we need Docker (same OS + same libraries) **and** seeds (same randomness).

Seeds + Docker solve the math and the machine. But you still need to remember what you did...

The Memory

Experiment Tracking with TrackIO

The Spreadsheet Graveyard

You've been tuning hyperparameters all week:

```
run_lr001_depth5_est100.py     → 87.2%
run_lr01_depth10_est200.py     → 91.4%  ← wait, was this the one?
run_lr01_depth10_est200_v2.py  → 89.1%  ← or this?
run_FINAL.py                   → ???
run_FINAL_FINAL.py             → ???
run_FINAL_USE_THIS.py          → definitely not this
```

You had **the** best model last Tuesday. Now you can't find it.

Seeds made results reproducible. But you didn't record what you did.

TrackIO: Three Calls Is All You Need

TrackIO (by Hugging Face) — free, local-first experiment tracking.

```
import trackio

# 1. Start a run
trackio.init(project="cs203-week08-demo", name="RandomForest",
             config={"model": "RF", "n_estimators": 100, "max_depth": 10})

# 2. Log metrics
trackio.log({"train_accuracy": 0.95, "test_accuracy": 0.845})

# 3. Finish
trackio.finish()
```

Everything stored locally in SQLite — no account, no cloud, no cost.

> **Demo**: `python trackio_1_training_curves.py`

TrackIO: Training Curves

Log metrics at each step → dashboard shows curves:

```
trackio.init(project="cs203-week08-demo", name="gb-training",
             config={"model": "GradientBoosting", "lr": 0.1})

for n_est in range(10, 310, 10):
    gb = GradientBoostingClassifier(n_estimators=n_est, learning_rate=0.1,
                                    max_depth=3, random_state=42)
    gb.fit(X_train, y_train)
    trackio.log({
        "n_estimators": n_est,
        "train_accuracy": float(round(gb.score(X_train, y_train), 4)),
        "test_accuracy": float(round(gb.score(X_test, y_test), 4)),
    })

trackio.finish()
```

Watch accuracy climb and plateau — just like TensorBoard, but simpler.

TrackIO: Comparing Runs

Run the same model with 3 learning rates → dashboard **overlays** them:

```
for lr in [0.01, 0.1, 0.5]:
    trackio.init(project="cs203-week08-demo", name=f"lr-{lr}",
                 config={"learning_rate": lr})

    for n_est in range(10, 210, 10):
        gb = GradientBoostingClassifier(n_estimators=n_est, learning_rate=lr,
                                        random_state=42)
        gb.fit(X_train, y_train)
        trackio.log({
            "n_estimators": n_est,
            "train_accuracy": float(round(gb.score(X_train, y_train), 4)),
            "test_accuracy": float(round(gb.score(X_test, y_test), 4)),
        })

    trackio.finish()
```

> **Demo**: `python trackio_3_compare_hyperparams.py`

TrackIO: What Did My Model Get Wrong?

Log **images** to see predictions, not just numbers:

```
trackio.init(project="cs203-week08-demo", name="prediction-analysis",
             config={"model": "RandomForest"})

# Create matplotlib grid of misclassified digits
fig = plot_misclassified_grid(X_test, y_test, preds)
fig.savefig("errors.png")

trackio.log({
    "misclassified": trackio.Image("errors.png",
        caption="All misclassified digits — red = wrong"),
    "test_accuracy": 0.97,
})
trackio.finish()
```

> **Demo**: `python trackio_2_misclassified.py` → check "Media & Tables" tab

TrackIO: Per-Class Breakdown Table

Log **tables** for structured analysis:

```
trackio.log({"per_class_metrics": trackio.Table(
    dataframe=pd.DataFrame(
        table_data,
        columns=["Digit", "Precision", "Recall", "F1", "Support"]),
)})
```

Digit	Precision	Recall	F1	Support
0	1.000	0.970	0.985	33
5	0.938	0.957	0.947	23
9	0.950	0.950	0.950	20

> **Demo**: `python trackio_4_per_class_table.py`

TrackIO: Alerts for Overfitting

Get notified when something goes wrong:

```
trackio.init(project="cs203-week08-demo", name="overfitting-detector",
             config={"model": "DecisionTree"})

for depth in range(1, 30):
    dt = DecisionTreeClassifier(max_depth=depth, random_state=42)
    dt.fit(X_train, y_train)
    gap = dt.score(X_train, y_train) - dt.score(X_test, y_test)

    trackio.log({"max_depth": depth, "overfit_gap": gap})

    if gap > 0.08:
        trackio.alert(title="Overfitting detected!",
                      text=f"Gap={gap:.1%} at depth={depth}",
                      level=trackio.AlertLevel.ERROR)
```

> **Demo**: `python trackio_5_overfitting_alert.py`

TrackIO: The Dashboard

```
pip install trackio
trackio show --project cs203-week08-demo
```

Tab	What You See
Metrics	Training curves, overlaid runs
Media & Tables	Images, per-class tables
Runs	All configs and final metrics
System Metrics	GPU/CPU usage (auto on Apple Silicon)

The Complete Reproducibility Stack

Layer	Tool	What It Controls
The Machine	Docker	OS + system libs + everything
The Packages	`venv` + `requirements.txt`	Library versions
The Math	`random_state=42`	Algorithmic randomness
The Memory	TrackIO	What you tried & what worked

```
Docker + Venv + Seeds + TrackIO = Time Capsule
```

You don't always need all layers. For course projects: seeds + TrackIO + venv is enough. Add Docker when shipping to production or sharing across OS.

Key Takeaways

1. **Docker ships the machine** — OS + libraries + Python + your code, frozen together
2. **Set seeds everywhere** — `random_state=42` in every sklearn call
3. **Track experiments** — `trackio.init()` / `log()` / `finish()` — never lose a good run
4. **Pin dependencies** — `pip freeze > requirements.txt` with exact versions
5. **Volumes for persistence** — containers have amnesia; use `-v` to save data
6. **Reproducibility is a gift to your future self** — if no one else can run it, it might as well not exist